

ASSIGNMENT: Module 4 – Lab: AWS Fundamentals (S3 & KMS)

Modern organizations increasingly rely on cloud platforms to store, process, and protect sensitive data. While cloud adoption improves scalability and cost efficiency, it also introduces new security responsibilities for customers. This lab explored several encryption methods available in Amazon Web Services (AWS), focusing on how data can be protected at rest and in transit using S3 and the Key Management Service (KMS). The exercises demonstrated server-side encryption with S3-managed keys (SSE-S3), server-side encryption with customer-managed KMS keys (SSE-KMS), client-side encryption using OpenSSL, and finally, the creation and use of an asymmetric KMS key for digital signatures. The goal of this lab was to understand not only how to configure these services, but also why encryption, key management, and auditing are essential components of secure cloud storage.

Lab Process:

The lab began by creating an AWS account and an IAM administrative user to avoid using the AWS root account for daily operations, following AWS best practices. After logging in as the admin user, I created the first S3 bucket with default encryption set to SSE-S3. Three test text files were generated in the Kali Linux VM and uploaded to the bucket. As shown in Appendix images 3–5, S3 automatically encrypted each object at rest using S3-managed keys.

In **Part 2**, I created a customer-managed symmetric KMS key and used it to configure a second S3 bucket with SSE-KMS. Uploading the same test files to this bucket demonstrated how S3 encrypts objects using the customer-managed key, and the S3 console displayed the key ARN associated with each object (Appendix images 6–11). This step highlighted the increased control and auditability provided by KMS.

Part 3 introduced client-side encryption. Using OpenSSL on the Kali Linux VM, I encrypted a file locally before uploading it to the SSE-S3 bucket. The encrypted file was then downloaded and decrypted manually using the same OpenSSL parameters and password. Appendix images 12–15 show the encryption, upload, download, and decryption workflow. This demonstrated layered encryption: S3 applied SSE-S3 to the ciphertext, while the underlying content remained encrypted with the client-side key.

In **Part 4**, I created an asymmetric RSA_4096 KMS key configured for Sign and Verify operations. After reviewing the key details and CloudTrail logs (Appendix images 16–18), I used the AWS CLI to sign a file hash on the lab5-signer EC2 instance. The signature and file were transferred via S3 to the lab5-verifier instance, where I downloaded the public key from KMS in .der format and converted it to .pem using OpenSSL to verify the signature. This completed the end-to-end asymmetric signing workflow.

Conceptual Understanding & Real-World Use

Asymmetric cryptography relies on a mathematically linked key pair: a private key used to create a digital signature and a public key used to verify it. Verification does not require access to the private key because the public key can mathematically validate the signature without exposing or interacting with the private key itself. This separation of roles is intentional and foundational to public key infrastructure (PKI). As Cloudflare explains, digital signatures allow anyone with the public key to verify authenticity and integrity while the private key remains protected (Cloudflare, n.d.). This aligns with the course reading’s emphasis on secure key management: “KMS is like a vault for your encryption keys... secure by design” (Sawant, 2025).

This model is widely used in real-world systems where trust, authenticity, and non-repudiation are essential. Legal, real estate, and corporate HR workflows are strong examples. Documents such as purchase agreements, mortgage disclosures, NDAs, employment contracts, and vendor

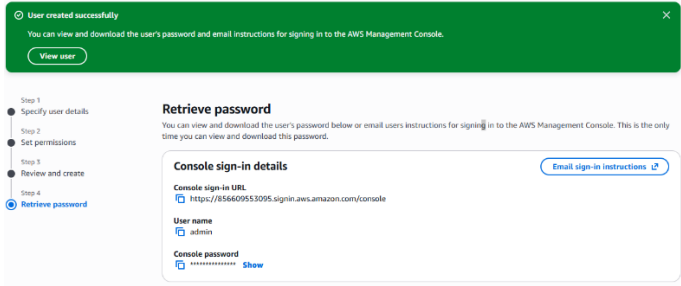
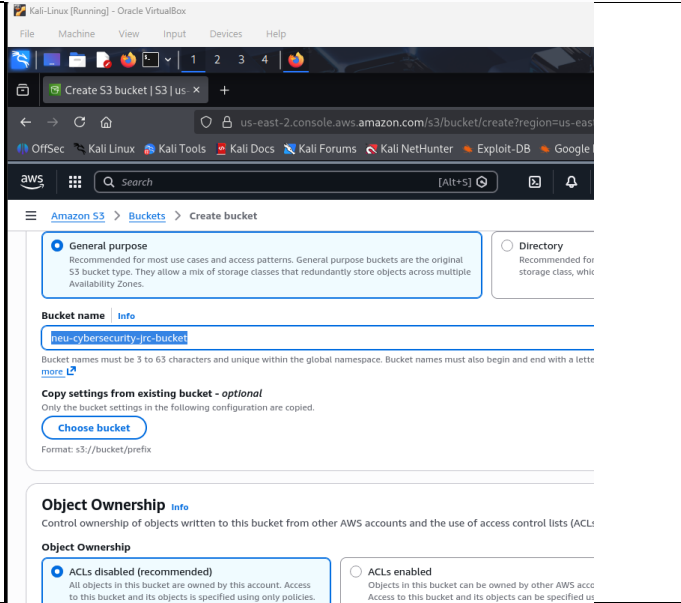
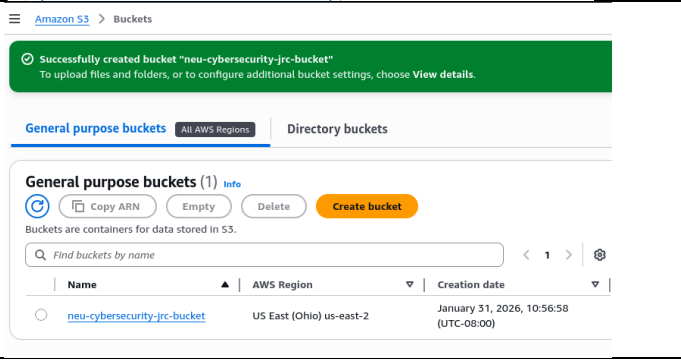
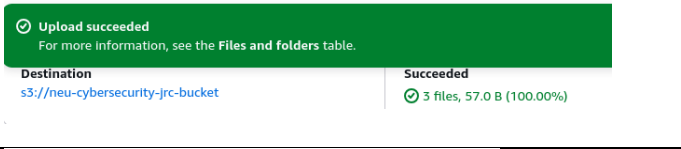
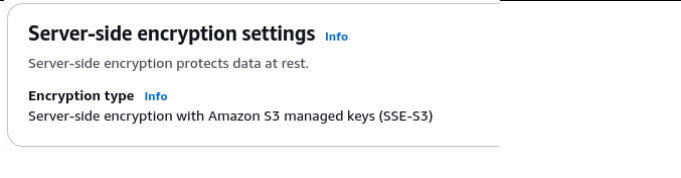
agreements must often be signed remotely and may pass through multiple systems. Without cryptographic protections, a malicious actor could attempt to alter a contract after it is signed, impersonate a signer, or substitute a different version of a document to gain advantage. Digital signatures prevent these attacks by binding the signer's identity to the exact version of the document that was signed; any modification invalidates the signature (DigiCert, n.d.).

Commercial platforms such as DocuSign and Adobe Sign rely on the same asymmetric cryptography demonstrated in this lab. Although these platforms provide a user-friendly interface, the underlying trust model is PKI-based. The signer's private key is stored in a secure hardware module controlled by the service, while the public certificate is embedded in the signed PDF. DocuSign notes that its digital signatures are backed by certificate authorities and tamper-evident seals that allow independent verification (DocuSign, n.d.). Even if the web interface were compromised, an attacker could not forge a valid signature without access to the private key stored in the service's secure infrastructure. Adobe similarly documents that certificate-based signatures allow recipients to verify authenticity and detect tampering using standard PDF readers (Adobe, n.d.).

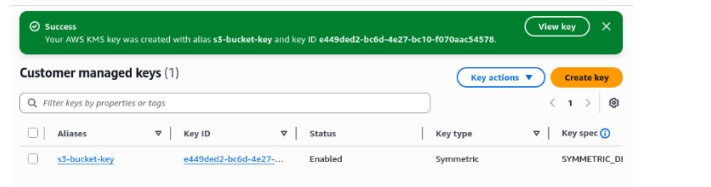
This mirrors the AWS KMS workflow used in the lab. The private key never leaves KMS, signing operations are logged in CloudTrail, and verification can be performed anywhere using only the public key. These characteristics—integrity, authenticity, non-repudiation, and independent verification—are precisely why asymmetric signatures are widely used in legal, financial, and corporate environments where trust must be established without exposing sensitive keys or relying solely on the security of a single application.

Together, these exercises demonstrated how AWS implements modern encryption practices and why strong key management, auditing, and digital signatures are essential for secure cloud operations.

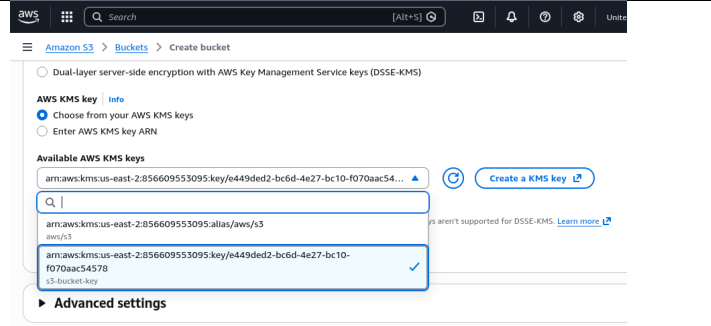
APPENDIX: Process Documentation

ACTIVITY DESCRIPTION	SCREEN CAPTURE
1. IAM user creation in AWS as “admin” This step happens after creating the AWS account while logged in as the root user. “Admin” user account will be used for day-to-day operations.	 The screenshot shows the AWS IAM console 'Create user' wizard. A green banner at the top states 'User created successfully'. Below, the 'Retrieve password' step is active, showing the console sign-in URL, user name 'admin', and a masked console password with a 'Show' link.
2. Bucket creation in AWS as neu-cybersecurity-jrc-bucket This happens inside the Kali-Linux VM, logged in as the admin user.	 The screenshot shows the 'Create bucket' page in the AWS S3 console. The 'General purpose' option is selected. The bucket name 'neu-cybersecurity-jrc-bucket' is entered. The 'Copy settings from existing bucket' section has a 'Choose bucket' button. The 'Object Ownership' section has 'ACLs disabled (recommended)' selected.
3. Verification of successful creation of first bucket in AWS After creating the S3 bucket, 3 text files were created on the Kali-Linux VM to be used for the exercise.	 The screenshot shows the 'Buckets' page in the AWS S3 console. A green banner at the top states 'Successfully created bucket "neu-cybersecurity-jrc-bucket"'. Below, the 'General purpose buckets' tab is active, showing a list of buckets with columns for Name, AWS Region, and Creation date. The newly created bucket is listed.
4. Verification of successful upload to S3 bucket	 The screenshot shows the 'Upload' progress bar in the AWS S3 console. A green banner at the top states 'Upload succeeded'. Below, the destination is 's3://neu-cybersecurity-jrc-bucket' and the status is 'Succeeded' with '3 files, 57.0 B (100.00%)'.
5. Confirmation of the SSE settings for upload of three test documents from the Kali-Linux VM showing “encrypted at rest” settings.	 The screenshot shows the 'Server-side encryption settings' page in the AWS S3 console. The 'Encryption type' is set to 'Server-side encryption with Amazon S3 managed keys (SSE-S3)'.

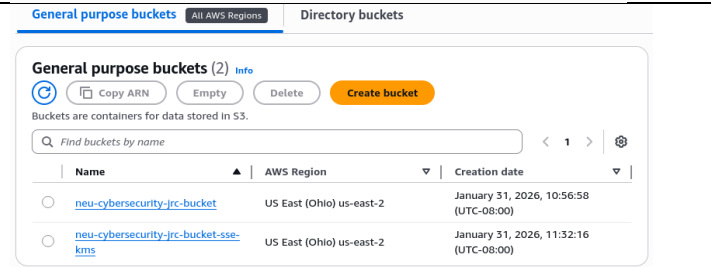
6. Navigated to KMS and created new key as symmetric/ encrypt & decrypt.



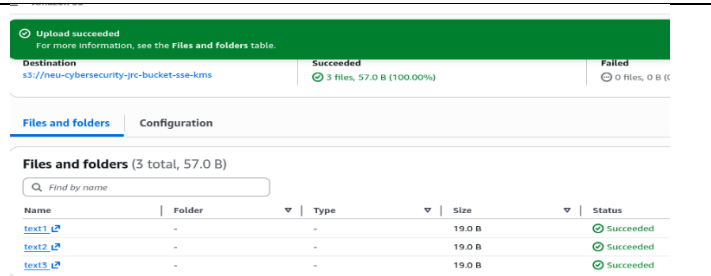
7. Created bucket AWS KMS key.



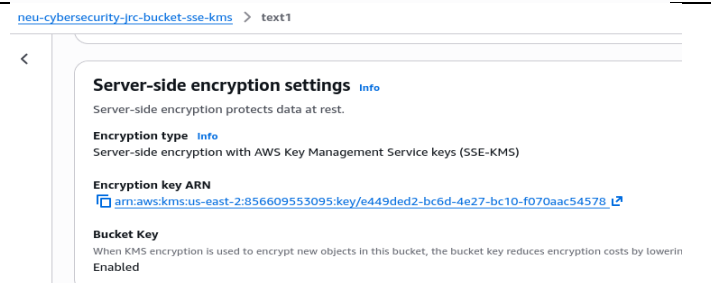
8. Screen showing both buckets created for lab.



9. Confirmation of upload of text files from Kali-Linux to sse-kms bucket.

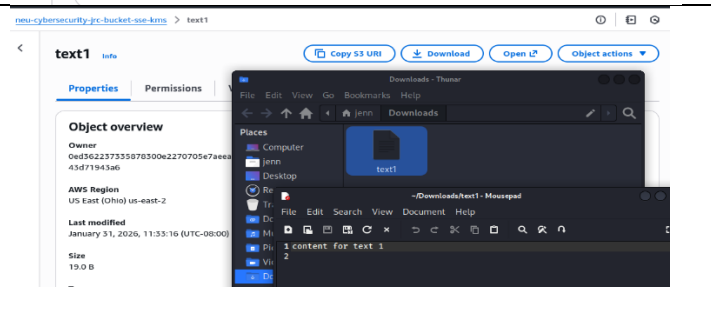


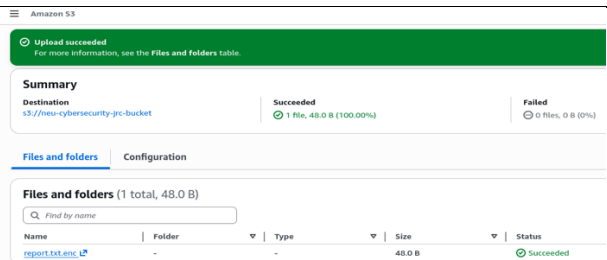
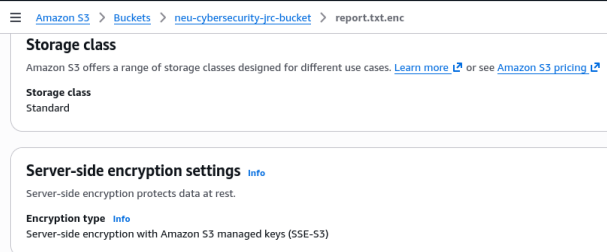
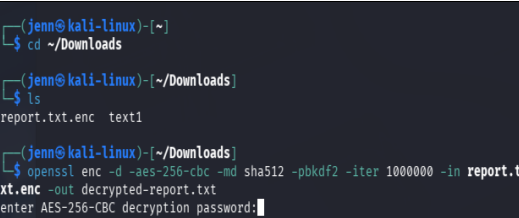
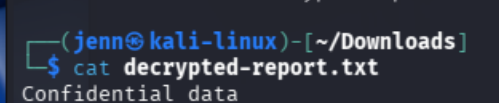
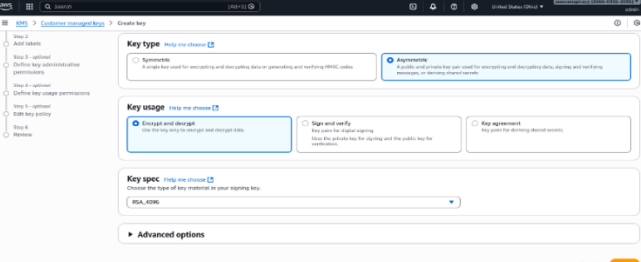
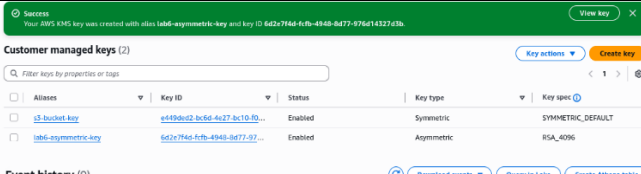
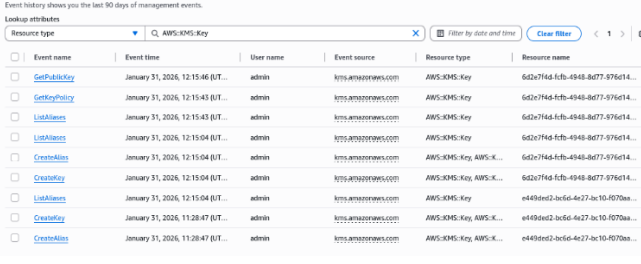
10. Confirmation of the encryption settings for one of the uploaded files.



11. S3 Object Properties Showing SSE-KMS Encryption Applied to text1

This screenshot displays the S3 console showing that the object “text1” is encrypted using SSE-KMS with customer-managed key. This confirms correct configuration of



server-side encryption with KMS in Part 2 of the lab.	
12. Confirmation of upload of encrypted file from Kali-Linux	
report.txt.enc was created using OpenSSL per instructions the more granular course document (see steps 44-52)	
13. Verification of the server side encryption settings for report.txt.enc (see step 54 in the class handout).	
14. After downloading the encrypted doc from AWS to Kali-Linux – we use OpenSSL to decrypt the file.	
15. Displaying the content of the decrypted file in Kali-Linux.	
16. Creation of an asymmetric RSA_4096 KMS key for Sign and Verify operations.	
17. confirmation screen for new asymmetric key (step 67).	
18. Confirmation of KMS Key event history prior to clean up tasks (see step 71).	
This was followed by download of public key in .der format from KMS and conversion to .pem using OpenSSL.	

METHODOLOGY:

This lab was completed using a structured, documentation-driven workflow grounded in the instructor-provided materials for Module 4. All technical steps—including creating S3 buckets, configuring server-side encryption, performing client-side encryption, generating KMS keys, and validating CloudTrail events—were executed manually in AWS and the Kali Linux VM. The primary procedural reference was the course document 4_Lab AWS Fundamentals (S3 and KMS) (Sawant, 2025), which I followed sequentially while capturing screenshots to document each major action. These screenshots were organized into an Appendix to provide verifiable evidence of correct configuration and adherence to the lab requirements.

Throughout the lab, I maintained a running log of actions, observations, and verification steps to ensure reproducibility and accuracy. This included noting when encryption settings were applied, when files were uploaded or downloaded, and when key-related events appeared in CloudTrail. This log informed the narrative sections of the report and ensured that the final documentation accurately reflected the work performed.

Copilot was used as a supplemental tool to support comprehension, organization, and writing clarity. Specifically, Copilot assisted in:

- (1) interpreting portions of the lab instructions,
- (2) rough draft review, refining the structure and clarity of the written report, and
- (3) identifying reputable external sources for background research.
- (4) generating reference list in APA format.

Copilot was not used to perform any technical steps in AWS, execute commands, generate screenshots, or produce outputs that were submitted as part of the lab's technical deliverables. All hands-on work—including configuration, encryption, signing, verification, and auditing—was completed independently. Copilot's role was limited to explanation, drafting, and editing support, consistent with academic integrity expectations and the course's guidelines for responsible AI use.

This combined approach—following the instructor's procedural guide, maintaining detailed process notes, and using Copilot for conceptual and editorial support—ensured that the lab was completed accurately, reproducibly, and with a clear understanding of both the technical workflow and the broader security principles demonstrated.

REFERENCES:

Adobe. (n.d.). Validate digital signatures. Adobe Support.

<https://helpx.adobe.com/acrobat/using/validating-digital-signatures.html>

Cloudflare. (n.d.). What is a digital signature? Cloudflare Learning Center.

<https://www.cloudflare.com/learning/ssl/what-is-a-digital-signature/>

DigiCert. (n.d.). What is a digital signature? DigiCert Blog. <https://www.digicert.com/blog/what-is-a-digital-signature>

DocuSign. (n.d.). How digital signatures work. DocuSign. <https://www.docusign.com/how-it-works/electronic-signature/digital-signature>

Sawant, A. (2025). 4_Lab AWS Fundamentals (S3 and KMS). Unpublished course document, Northeastern University.

Sawant, A. (2025). Module 4: Secure storage and retention. Coursera course reading for DAMG 7350, Northeastern University.